DATA STRUCTURES – ASSIGNMENT 2

Searching in Linked Lists and Mobile Contact Applications

Name	M Prince Daniel	**Register Number**	1972260073B

Question 1: Can binary search be used on a linked list? Explain the challenges and potential solutions.

Answer: Yes, binary search can be used on a linked list if the elements are sorted. However, it is not efficient compared with binary search on an array.

Challenges:

1. No direct access: In an array, we can access the middle element directly using its index. In a linked list, we must move node by node from the head to reach the middle.

2. Extra traversal: Finding the middle node requires traversing the list. This process is repeated during each step of binary search.

3. Reduced efficiency: Although binary search takes O(log n) comparisons on an array, locating middle nodes in a linked list takes additional time. A straightforward linked-list implementation can take O(n log n) time, so linear search with O(n) time is usually better.

Potential solutions:

• Use a slow and fast pointer technique to find the middle node of a linked list.

• If frequent searching is required, store the data in an array or another structure that supports fast random access.

• Use an auxiliary index or a skip list when the application needs faster searching while retaining linked data structures.

Conclusion: Binary search is possible on a sorted linked list, but it is generally not preferred because linked lists do not support direct access to the middle element. Linear search is usually simpler and more efficient for a normal linked list.

Question 2: In a mobile contact app, how would you implement searching? Justify whether linear or binary search fits better.

Answer: In a mobile contact application, searching can be implemented by comparing the name or phone number entered by the user with the saved contacts.

Linear search: It checks contacts one by one until the required contact is found. It works even when contacts are not sorted. Its time complexity is O(n).

Binary search: It repeatedly divides a sorted list of contacts into two halves and searches the appropriate half. Its time complexity is O(log n), provided the contacts are sorted and random access is available.

Which is better?

For a large contact list that is maintained in sorted order by name, binary search is generally better because it needs fewer comparisons and can locate a contact faster. When a new contact is added or deleted, the list must remain sorted for binary search to work correctly.

For a small or unsorted contact list, linear search is easier to implement and does not require sorting. A practical contact app may also use an indexed search structure or prefix-search technique to show matching names while the user types.

Conclusion: Binary search is a better choice for searching a large, sorted contact list. Linear search is suitable for small or unsorted lists. The choice depends on the number of contacts and how the data is stored.